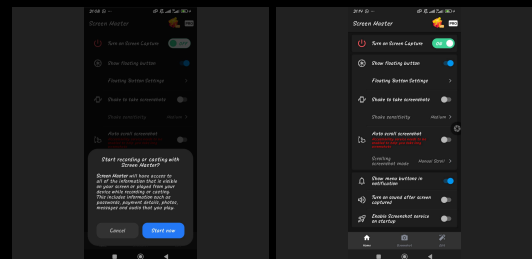


Tuesday 9:18 PM



This project for my personal app building. Idea is to make, lots of unknown reels about films and tv serious in social medias like inster, fb, Youtube, Tiktok. I want to make a app to find it very quickly. This app can be use background like shazam. My idea is Open the app and start the app like screen master app, then small Bubble bar show in the side.(2nd image showing that small bubble) This bubble bar needs to be the key of the app. Reel identify using small screen recording.

Also i have idea lots of ai videos in social media. easily people can find this as a ai video or not. can we do this with attractive output. Can we create this app.

if this succeeded i will publish in play store and make some small money.

First we need research right?

> What is your initial development budget?

Nearly zero—free services only

Show less

Worked for 1m 1s

Yes, this app can be created. The concept is technically feasible and commercially interesting, but we should build it in carefully controlled stages.

The strongest version of your idea is:

While someone watches an unknown movie/TV-series Reel, they tap the Reelix floating bubble. Reelix captures a short sample, analyses the visual frames, dialogue, subtitles and available provenance data, then displays the most likely movie or series with a confidence score.

1. What the user experience could look like

1. User opens Reelix.
2. Taps **Enable Reel Scanner**.
3. Android displays the official screen-capture permission dialog like your first screenshot.
4. A small Reelix bubble appears on the side of the screen like your second screenshot.
5. User opens Instagram, TikTok, Facebook or YouTube.
6. When an unknown scene appears, the user taps the bubble.
7. Reelix captures approximately 5–8 seconds.
8. The bubble changes into a scanning animation.
9. A small result card appears:

STRONG MATCH • 91%

Stranger Things

Season 2 • Episode 6

2016 • Netflix

Matched using:

- ✓ Character appearance
- ✓ Dialogue
- ✓ Subtitle text

AI-content check:

Likely authentic production footage

Confidence: 76%

The result could include:

- Movie or series name
- Poster and release year
- Probable episode
- Actor or character names

- Short description
- Identification confidence
- “Why we think this” explanation
- AI-generated-video assessment
- Where to watch
- Save to watchlist
- “Wrong result?” feedback button

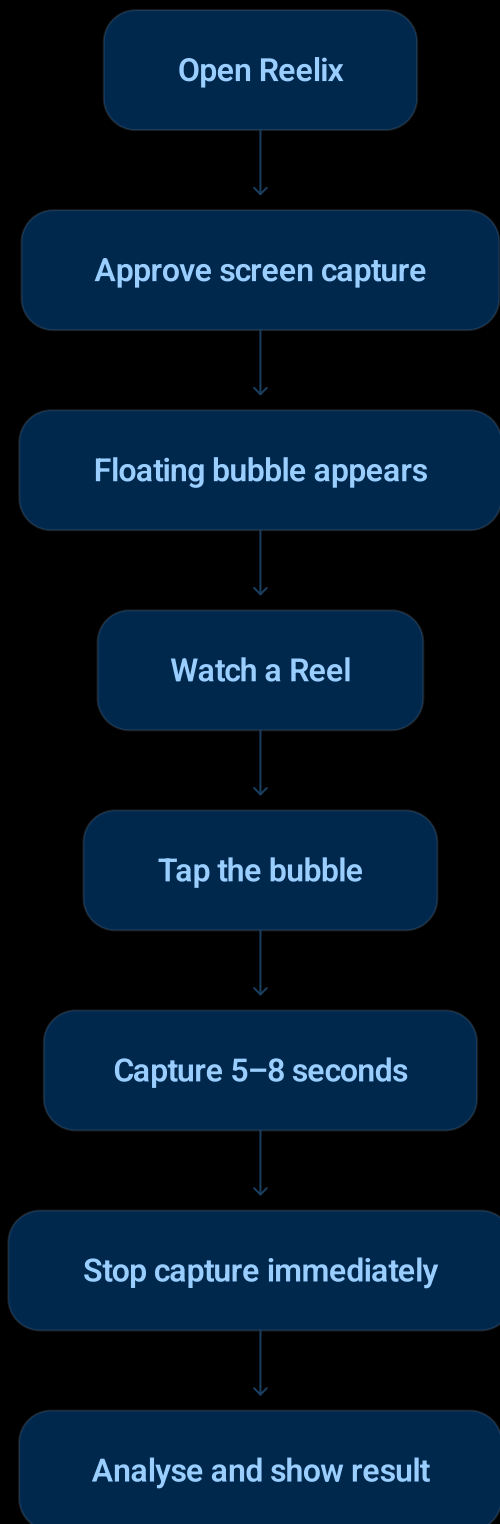
That would produce the attractive, transparent output you are imagining.

2. Important Android limitation

The floating bubble is possible on Android using an application overlay. Screen capture is possible through Android’s `MediaProjection` API.

However, modern Android does not allow Reelix to silently record everything forever. For applications targeting Android 14 or newer, Android requires user consent for each screen-capture session, and screen capture must operate through an appropriate foreground service.  developer.andr... +1

Therefore, the realistic experience is:



It should not secretly monitor the user's screen. That would create privacy, security, battery and Play Store compliance problems.

We should also avoid depending on Android Accessibility Service unless it is genuinely required. Google Play applies special disclosure and policy requirements to Accessibility API use.  Play Console ... +1

3. How Reelix would identify the movie

One technique will not be reliable enough. Reelix should combine several signals.

Identification signal	What Reelix extracts	Example
Visual matching	3–6 clean frames	Actors, locations, clothing and cinematography
Dialogue matching	Speech converted into text	“We have to get out of Hawkins”
Subtitle/OCR matching	Text displayed inside the Reel	Captions or original subtitles
Audio fingerprinting	Recognizable soundtrack/audio pattern	Original scene audio
Metadata/provenance	File credentials or source metadata	Creator/tool information
Context reasoning	Combination of all evidence	Actor + quote + location

A practical confidence calculation could be:

$$FinalScore = 0.40V + 0.30D + 0.15O + 0.10A + 0.05M$$

Where:

- V = visual-match score
- D = dialogue-match score
- O = OCR/subtitle score
- A = audio-match score
- M = metadata score

These weights would later be improved using real test data.

The hardest part

The mobile app and floating bubble are not the hardest components. The real challenge is creating or accessing a searchable database of movie and TV scenes.

A recognition engine must compare the Reel against something. TMDB can provide posters, titles, cast and descriptive metadata, but it does not provide a searchable collection of complete movie footage. TMDB’s API is free only for non-commercial use with attribution; a commercial application requires discussing licensing with TMDB.

We must not build the database by downloading copyrighted movies or scraping streaming platforms without permission.

4. Near-zero-cost recognition strategy

For the prototype, we can avoid maintaining a massive scene database.

Stage A: Capture and clean the sample

The phone will:

- Capture 5–8 seconds
- Crop social-media interface elements
- Select approximately 4 useful frames
- Compress the frames
- Extract visible subtitle text
- Extract a short audio segment
- Delete the raw recording after processing

Stage B: Perform multimodal identification

Send only the selected evidence to the identification service:

JSON

```
{
  "frames": ["frame1", "frame2", "frame3"],
  "dialogue": "Recognized dialogue here",
  "visibleText": "Subtitle text here",
  "sourceApp": "Instagram"
}
```

A multimodal model can suggest several possible titles. A free Gemini API tier can support prototype experimentation, although free quotas and pricing can change and therefore should not be treated as unlimited production infrastructure.  ai.google.d...

Stage C: Verify the result

Reelix should not accept the first AI answer blindly. It should:

1. Generate up to five candidates.
2. Retrieve candidate metadata.
3. Compare actors, plot terms, year and visual evidence.
4. Calculate a confidence score.
5. Return a result only when confidence is sufficient.

If confidence is low, it should say:

“We found two possible matches”
rather than presenting an uncertain answer as a fact.

5. AI-generated-video detection

This feature is possible, but it cannot honestly provide a guaranteed “Real/Fake” verdict.

Current research shows that AI-video detectors often struggle when tested against videos created by generators not represented in their training data. Challenging benchmarks also show limited performance on sophisticated and realistic synthetic videos. [arxiv.org +1](#)

Therefore, the result must be framed as an assessment:

- **Verified AI-generated**
- **Likely AI-generated**
- **Mixed or edited**
- **No strong AI evidence**
- **Unable to determine**

It should never say “100% real.”

Recommended AI-analysis pipeline

Reelix can use four evidence layers:

1. Content Credentials

Check whether the file contains C2PA Content Credentials showing its origin or editing history. Open-source C2PA tools are available for inspecting and validating manifests. [opensource.co... +2](#)

2. Metadata

Inspect creator-tool names, encoding information and editing history where available.

3. Visual-temporal analysis

Examine inconsistent motion, changing object geometry, unstable text, lighting inconsistencies and frame-to-frame abnormalities.

4. Audio-visual consistency

Evaluate whether voice, lip movement, environmental sound and visual actions agree.

Because social platforms may recompress videos or remove provenance metadata, an absent C2PA record does not prove that a video is real.

Attractive AI result

AI CONTENT ANALYSIS

Likely AI-generated • 82%

Evidence:

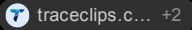
- Objects change shape between frames
- Background text is temporally unstable
- Motion pattern resembles generated video
- No verifiable Content Credentials found

Important:

This is an automated assessment, not absolute proof.

This explanation is much more trustworthy than showing only “82% fake.”

6. Existing competition

The market is not empty. Products such as Trace already promote movie/TV clip identification and exact-scene matching, while WMIT offers identification through screenshots, clips, links and descriptions.  +2

That does not mean we should stop. It means Reelix requires clear differentiation.

Your potential competitive advantage is the combination of:

- Android floating bubble
- One-tap short screen capture
- Visual + dialogue + subtitle identification
- Movie and TV-series support
- AI-generated-video assessment
- Transparent evidence and confidence
- Fast result without manually saving and uploading a Reel
- Sinhala-friendly/simple interface later

The floating bubble plus AI-authenticity explanation could become Reelix’s main product identity.

7. Recommended MVP scope

We should not build every feature initially. The first MVP should validate only the critical assumption:

Can Reelix correctly identify enough social-media movie clips to make users return?

MVP 1 — Proof of concept

Build a simple upload-based prototype:

- Upload a screenshot
- Upload a 5–8 second clip
- Extract frames
- Extract subtitles/dialogue
- Return top-three movie or series matches
- Display confidence and evidence
- Let the tester mark correct/incorrect

No login, payment, floating bubble or advertisements yet.

MVP 2 — Android capture experience

After the recognition quality is acceptable:

- Enable screen-capture session
- Show draggable floating bubble
- Capture a short sample
- Show scanning progress
- Display an overlay result card
- Store identification history

MVP 3 — AI-content assessment

Then add:

- C2PA verification
- Metadata inspection
- Basic visual detector
- Evidence-based AI likelihood
- User warning about uncertainty

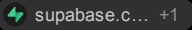
MVP 4 — Commercial application

Finally add:

- Accounts and cloud synchronisation
- Watchlist
- Streaming-platform links
- Free daily scans
- Rewarded advertisements
- Premium unlimited scans
- Analytics and abuse protection

8. Suggested free-first technology stack

Layer	Recommended technology
Mobile application	Flutter
Android-specific capture/bubble	Kotlin platform channel
Local video processing	FFmpeg Kit alternative/native MediaCodec and OpenCV
OCR	Google ML Kit on-device text recognition
Speech-to-text prototype	Whisper locally on development machine
Visual embeddings	OpenCLIP
Similarity search	FAISS initially; pgvector later
Prototype AI reasoning	Gemini free tier
Backend	FastAPI or Node.js
Database/auth	Supabase free plan
Content Credentials	Open-source C2PA SDK
Movie metadata	TMDB during non-commercial prototype, with attribution

Supabase currently offers two active free projects, which is enough for early development, but video files should not be permanently stored because free storage and bandwidth are limited.  +1

9. Privacy requirements

Because Reelix captures another application's screen, privacy must be treated as a core product requirement.

The design should enforce:

- Capture begins only after an explicit tap.
- A permanent notification appears during screen capture.
- Capture stops automatically after a few seconds.
- Raw clips are deleted after analysis.
- Frames are encrypted during transmission.
- Payment, password and private-message screens are never intentionally analysed.
- The user receives a clear disclosure before enabling capture.
- The Play Store Data Safety form accurately explains collection and processing.

- Users can delete their history and uploaded evidence.
- No footage is used for model training without separate consent.

This will reduce both user distrust and Play Store review risk.

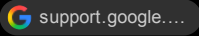
10. Can it make money?

Potentially yes, but recognition cost must remain below revenue per scan.

A sensible model is:

- 3–5 free scans daily
- Rewarded advertisement for additional scans
- Low-cost monthly Pro plan
- Affiliate streaming links where permitted
- No intrusive advertisement inside the floating result
- AI analysis available as a premium or limited feature

Google currently charges a one-time **US\$25 Play Console registration fee**, so the publication stage will not be completely free. 

Also, if Reelix sells digital subscriptions through Google Play, Google's applicable service fee must be included in the financial model. Google currently describes a 15% tier for the first US\$1 million of annual earnings for enrolled developers. 

Final feasibility assessment

Area	Assessment
Floating Android bubble	Feasible
User-triggered short screen capture	Feasible
Screenshot-based identification	Feasible for an MVP
Dialogue + subtitle matching	Feasible
Exact episode identification	Possible, but significantly harder
Identification of every movie ever made	Not realistic initially
AI-video assessment	Feasible as probability/evidence, not certainty
Completely free production service	Not sustainable at scale
Near-zero-cost prototype	Realistic

Area	Assessment
Play Store publication	Realistic with careful privacy compliance
Commercial opportunity	Possible, but recognition accuracy must be validated first

My recommendation is to proceed, but treat **movie/series identification as the core MVP** and **AI-video detection as the second major feature**. The first concrete deliverable should be a research-and-validation specification covering competitors, users, architecture, datasets, legal constraints, costs and a prototype test plan.

Does this absolutely require a dedicated database? That wouldn't be free to set up. How are other similar apps built?

Also, as I mentioned, I need to record the screen, but the recordings shouldn't be saved on the users' phones—otherwise, their storage would fill up.

U said in realistic experience chart is the exact what i need to implement

Worked for 32s

Yes—the chart is exactly the interaction flow Reelix should implement.

And no, Reelix does **not absolutely require an expensive dedicated database for the first version**. But we need to distinguish between two different things:

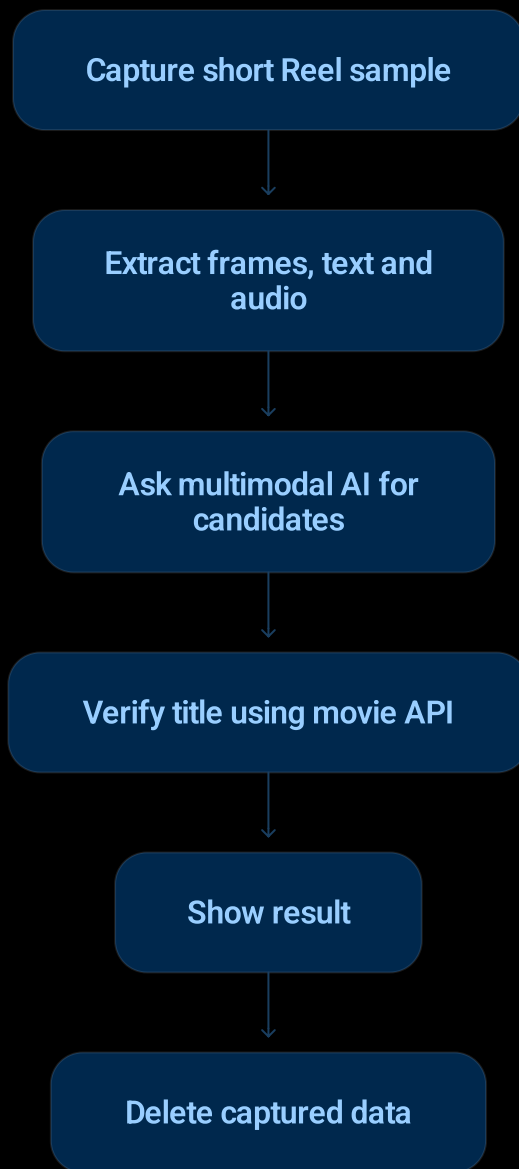
1. A normal application database
2. A movie-recognition index

They are not the same.

1. Does Reelix require a database?

For the prototype: No dedicated paid database

The first prototype can work like this:



For this version:

- Gemini or another multimodal model identifies possible movies.
- TMDB provides title, poster, cast and description.
- Recognition history can initially remain locally on the phone.
- No video collection is maintained.
- No custom movie-scene database is required.
- No paid server is required during development.

This is the best approach for validating the idea with almost zero budget.

For a large, accurate production app: Some searchable index is eventually necessary

If you want Reelix to identify the **exact movie, episode and scene reliably**, the system needs previously indexed reference information.

For example:

Captured Reel fingerprint:

A72-B91-X33

Search index:

Movie 1 → D12-R43-P99

Movie 2 → A72-B91-X33 ← Match

Movie 3 → J65-T11-L20

The database does not need to contain full movie files. It could contain only:

- Frame embeddings
- Audio fingerprints
- Dialogue fragments
- Subtitle fragments
- Movie and episode IDs
- Timestamps
- Poster/metadata references

These fingerprints are much smaller than the original movies.

Video-fingerprinting systems work by comparing a query fingerprint against a prebuilt collection of fingerprints.  researchgate.... +1

Therefore, the honest answer is:

Reelix stage	Dedicated recognition database
UI prototype	Not needed
AI identification prototype	Not needed
First small beta	Optional
Reliable exact-scene identification	Needed eventually
Large commercial platform	Definitely needed

2. How similar apps are probably built

Applications such as Trace and WMIT do not publicly disclose their full backend architecture, so we should not pretend to know their proprietary implementation.

However, based on how video-recognition systems work, similar apps usually use one or a combination of these approaches.

Approach A — Multimodal AI

The app sends:

- Screenshots
- Dialogue
- Subtitle text
- Scene description

A multimodal AI model tries to recognize the content from its existing knowledge.

Advantages:

- No movie database required initially
- Quick to build
- Good for popular content
- Suitable for a prototype

Disadvantages:

- Can confidently return the wrong answer
- May not know new or uncommon content
- Exact episode identification is difficult
- API usage eventually costs money

Approach B — Visual or audio fingerprint database

The company processes films beforehand and generates small fingerprints for different scenes.

When the user scans a Reel:

1. The Reel is converted into a fingerprint.
2. That fingerprint is searched against the reference index.
3. The closest movie and timestamp are returned.

Historically, similar movie-recognition systems have combined audio recognition, computer vision and a database of indexed movie scenes. One documented system initially launched with a controlled collection of more than 100 movies and expanded it gradually.

 wired.c...

Advantages:

- Much more accurate for indexed films
- Can identify the exact scene
- Fast after the index is prepared
- Less dependent on an expensive AI model per request

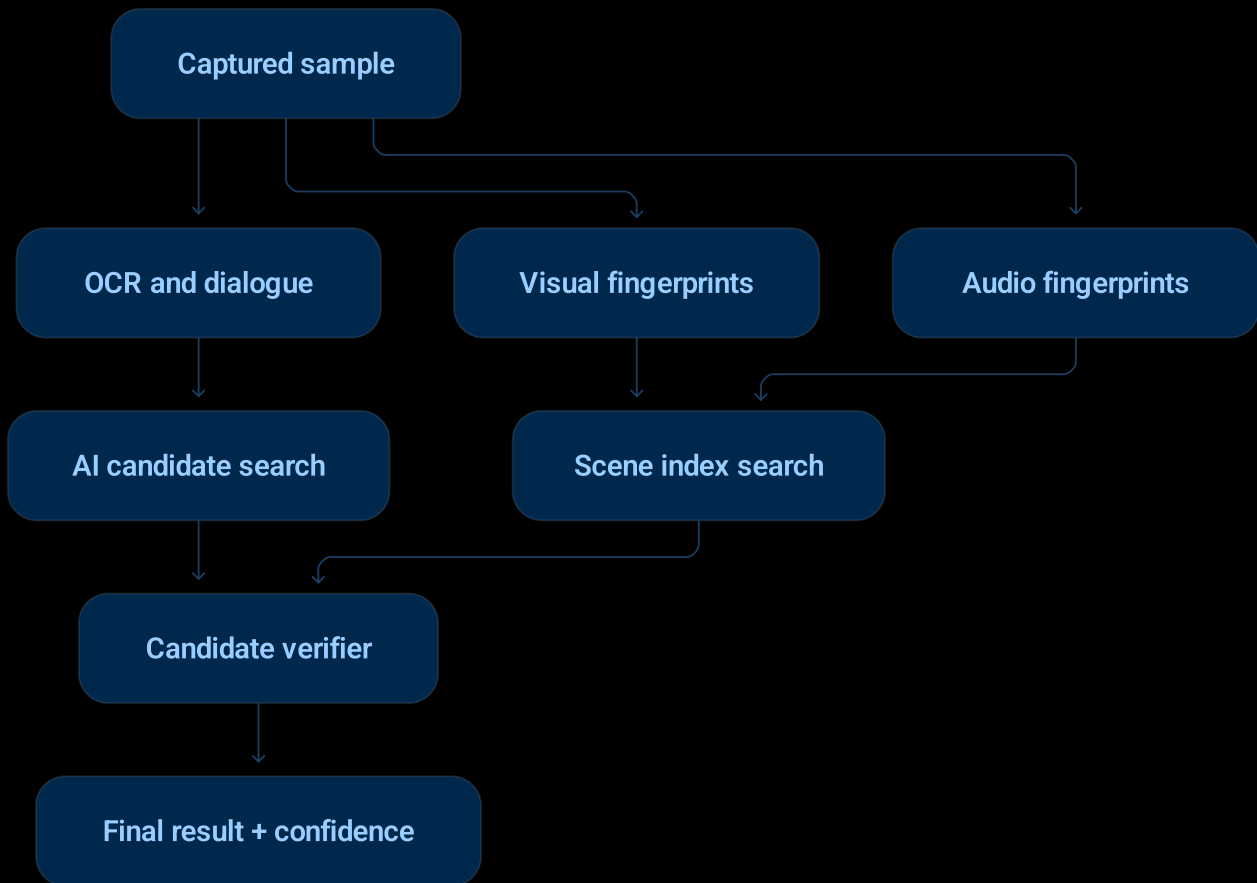
Disadvantages:

- The content must be indexed first

- Copyright and licensing must be handled properly
- Processing thousands of films needs computing resources
- Storage and search infrastructure eventually cost money

Approach C — Hybrid identification

This is the strongest architecture and the one I recommend for Reelix:



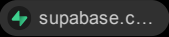
Initially, the scene-index branch can be absent. We can introduce it after validating demand.

3. Free database options

Even when we eventually need structured data, “database” does not automatically mean paying money.

During development, we can use:

- SQLite locally
- PostgreSQL locally on your computer
- FAISS for local vector similarity search
- Supabase free tier
- A simple JSON file for early testing
- On-device Room/SQLite for user history

Supabase currently provides two active projects on its free plan, which is enough for an early Reelix prototype. 

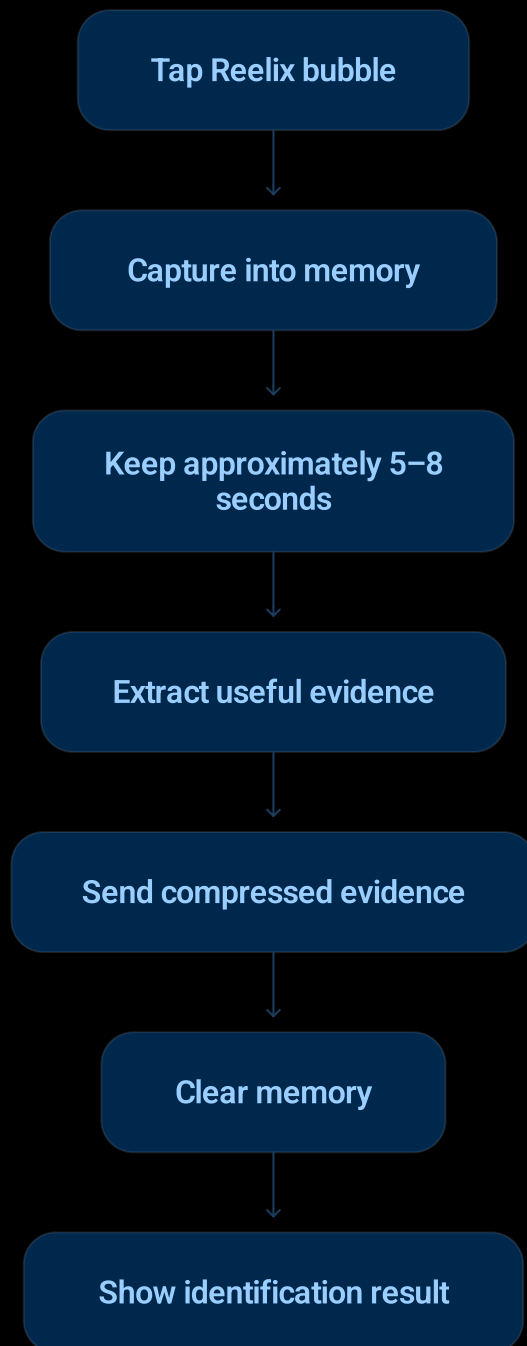
A practical structure would be:

Data	Initial storage
User settings	On-device storage
Scan history	On-device SQLite
Movie posters/details	Retrieved from API and cached
Temporary captured frames	RAM only
Raw recording	Not permanently stored
Recognition candidates	Returned by AI
User feedback	Free backend later
Movie fingerprints	Added in a future phase

4. The recording does not need to appear on the phone

You are completely correct: Reelix should not save captured clips to the user’s Gallery, Downloads or Videos folder.

The screen-capture flow should be designed as a temporary processing pipeline.



Android's `MediaProjection` provides Reelix with a screen-capture stream. It does not force the application to save a normal video into the Gallery. [API refere...](#)

We control where the captured data goes.

Recommended Reelix method: Capture evidence, not a video file

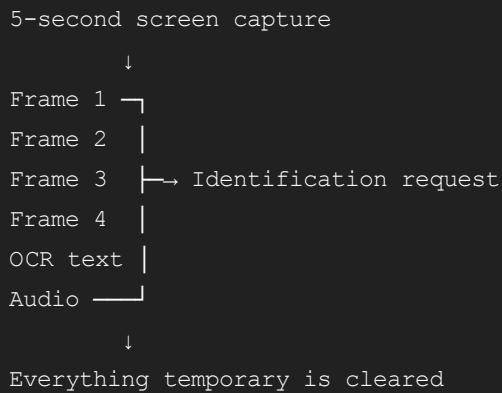
Reelix does not actually need to create a playable `.mp4` recording.

During the 5–8 second scanning period, Reelix can:

1. Receive screen frames in memory.
2. Select one useful frame every 1–2 seconds.
3. Keep approximately 4–6 compressed images.
4. Run OCR on those frames.

5. Capture a short audio buffer when Android permits it.
6. Send the selected evidence for analysis.
7. Release all memory when processing finishes.

For example:



This approach offers several operational benefits:

- No recording appears in the Gallery.
- User storage does not gradually fill.
- Much less internet data is used.
- Analysis is faster.
- Privacy exposure is reduced.
- The server does not receive unnecessary footage.

5. What if temporary files are required?

Some Android encoders or Flutter libraries may work more reliably with a temporary file. If that occurs, Reelix can write into its private cache directory—not the Gallery.

Example:

```
/data/user/0/com.reelix.app/cache/scan_123.tmp
```

Then:

1. Create the temporary file.
2. Process or upload it.
3. Delete it immediately.
4. Delete any abandoned cache files when Reelix starts again.

Android documentation says applications should actively maintain and remove their own cache files instead of depending entirely on Android to clean them.  App data and fi...

Therefore, Reelix should have two cleanup protections:

Normal completion → Delete temporary capture immediately
App crash/restart → Delete every abandoned scan file

The file would remain invisible to the user's Gallery and would exist only briefly inside Reelix's private storage.

6. Can audio always be captured?

Not necessarily.

Android supports capturing audio played by other applications, but the source application can disable playback capture. Android applications targeting Android 10 or newer generally permit capture by default, though developers can explicitly turn it off.

 Android media

That means:

- Instagram audio may work on some devices/configurations.
- Some protected streaming applications may block it.
- DRM-protected video may display a black screen.
- Reelix must still work using frames and OCR when audio is unavailable.

Our pipeline should therefore degrade gracefully:

Frames + OCR + audio → Strongest analysis
Frames + OCR → Normal analysis
Frames only → Basic analysis
Black/protected view → Cannot capture this content

Reelix should not attempt to bypass protected-content restrictions.

7. Exact Reelix experience to implement

Your final user flow should be:

Step 1 — Start Reelix

The home screen contains:

- **Start Reel Scanner**
- Floating bubble toggle
- Privacy explanation
- Previous results
- Settings

Step 2 — Accept Android permission

Android displays its official screen-capture warning.

The user taps **Start now**.

This consent is required on modern Android for the capture session. Reelix cannot silently grant itself permission. Android 14 and newer require consent for each `MediaProjection` capture session.  developer.andr...

Step 3 — Display the floating bubble

The application minimizes and displays the Reelix bubble over Instagram, TikTok, Facebook or YouTube.

Possible bubble states:

State	Appearance
Ready	Purple Reelix logo
Capturing	Red animated ring
Analysing	Blue rotating animation
Match found	Green check
No match	Orange question mark

Step 4 — User taps the bubble

Reelix should:

- Hide the bubble temporarily so it is not captured inside the frames.
- Capture approximately 5 seconds.
- Select useful frames.
- Extract subtitle text.
- Capture permissible audio.
- Stop immediately.
- Bring the bubble back.

Step 5 — Analyse without saving

The extracted evidence is analysed from memory or private temporary cache.

No user-visible recording is created.

Step 6 — Show a compact overlay result

REELIX MATCH

Interstellar
2014 • Science Fiction

Match confidence: 92%

Matched from:

- ✓ Visual scene
- ✓ Recognized actor
- ✓ Subtitle context

[View details] [Save] [Wrong result]

The user can continue scrolling without opening the full Reelix application.

8. Best implementation strategy

We should not begin with the complicated floating bubble immediately. First, we need to prove the recognition engine.

Phase 1 — Recognition experiment

Build a simple screen where you manually select a screenshot or short clip.

Test approximately:

- 50 popular movie Reels
- 30 TV-series Reels
- 20 difficult or edited Reels

Measure:

- Correct title in first result
- Correct title in top three results
- Analysis time
- Failure reasons
- Internet data consumed

Suggested success threshold:

Metric	MVP target
Popular movie top-1 accuracy	At least 75%
Top-3 accuracy	At least 90%
Average response time	Under 10 seconds

Metric	MVP target
Temporary files remaining	0
Incorrect “high confidence” results	Under 5%

Phase 2 — Build the Android capture service

After Phase 1 works:

- MediaProjection permission
- Foreground capture service
- Floating overlay
- RAM frame buffer
- OCR
- Automatic cleanup

Because you are already familiar with Flutter, we can develop the interface in Flutter and implement the capture service and overlay using native Kotlin through a Flutter platform channel.

Phase 3 — Improve accuracy

Only after real testing should we decide whether Reelix needs:

- A small fingerprint database
- Dialogue/subtitle search
- A vector index
- User-contributed corrections
- Content partnerships

Final decision

You do **not** need to purchase or set up a large dedicated database now.

The recommended first architecture is:

```
Flutter application
+
Native Kotlin screen capture
+
Memory-only frame extraction
+
On-device OCR
+
Free-tier multimodal AI
```

No recordings are permanently saved on the phone. Only a few temporary frames and optional audio samples are processed, then deleted.

Later, if Reelix receives users and needs better exact-scene accuracy, we can introduce a small legal fingerprint index gradually. That investment should happen only after the MVP demonstrates real product-market demand.

Wednesday 12:06 AM

For the prototype: No dedicated paid database

i didnt ask u about the prototype. im asking about the full app

You’re correct—I answered about the prototype when you were asking about the finished, published app.

For the **full Reelix app**, the direct answer is:

Reelix will require backend infrastructure and some persistent databases, but it does not need to store screen recordings or full movies.

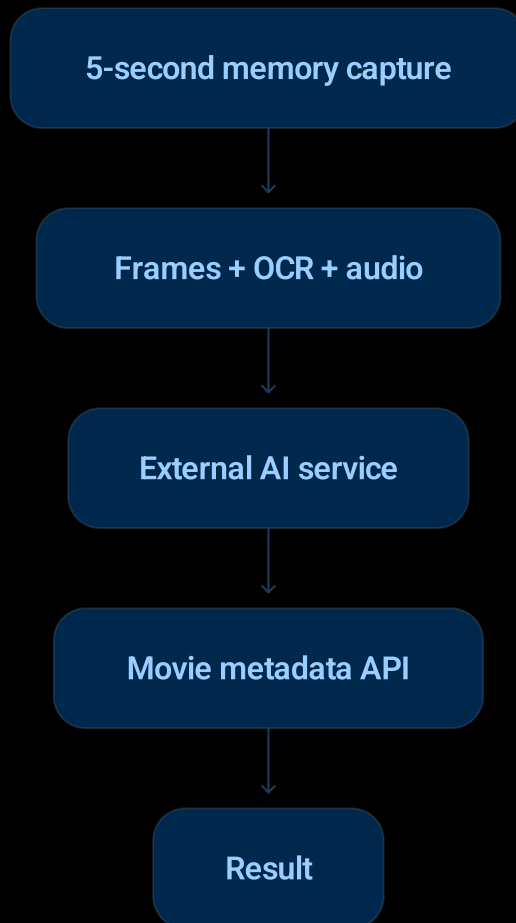
What the full app actually needs

There are two separate database requirements.

Database	Purpose	Required?
Application database	Users, scan history, saved movies, subscriptions, feedback	Yes
Recognition index	Movie/series fingerprints, dialogue, frames, episode and timestamp mapping	Yes, unless Reelix permanently depends on a third-party recognition service
Screen-recording storage	Store captured Reels	No
Full movie storage	Store complete movies	No—and this creates licensing problems

A full commercial Reelix therefore has two possible operating models.

Option 1: Depend on external AI services



The external AI provider effectively supplies the recognition intelligence. Reelix only keeps:

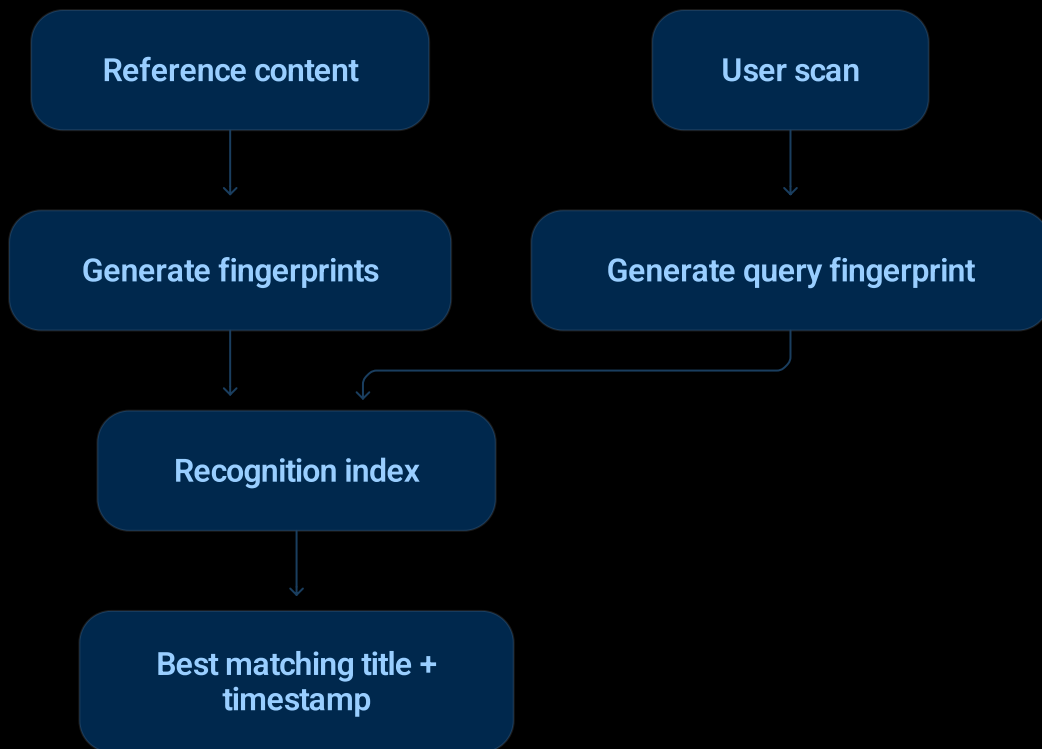
- Accounts
- Scan results
- Watchlists
- Subscriptions
- User corrections

This avoids building a huge movie index, but every scan consumes an API quota. Once Reelix has many users, it will not remain free.

It can also create:

- Dependency on the AI provider
- Changing API prices
- Rate limits
- Incorrect or hallucinated results
- Poor exact-episode recognition
- Privacy considerations when sending captured evidence

Option 2: Build Reelix's own recognition index



This is how a real “Shazam for movies” system becomes fast and reliable.

But the index stores only mathematical representations such as:

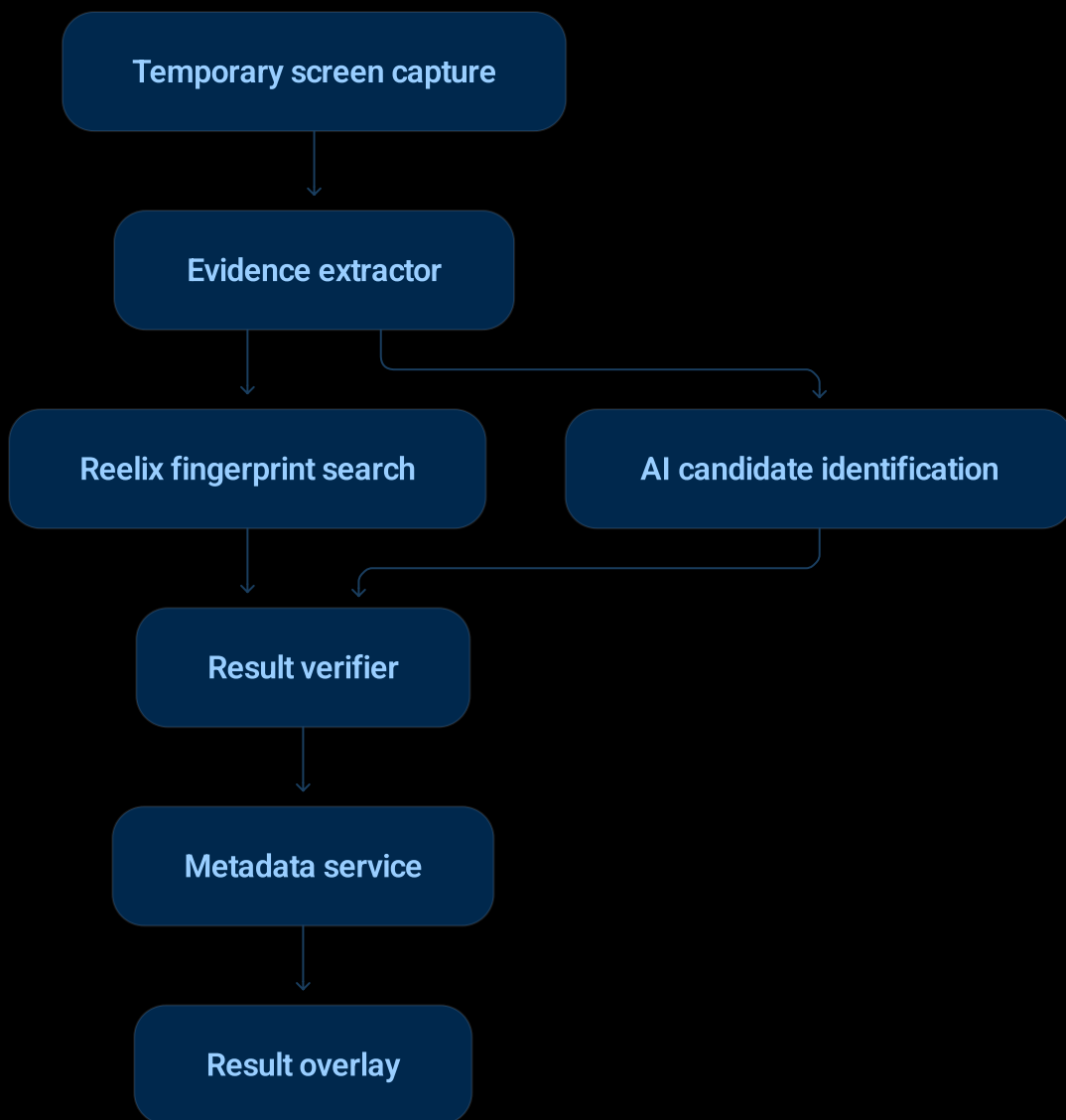
```
Title: Interstellar
Timestamp: 01:24:16
Visual vector: [0.018, -0.274, 0.391, ...]
Audio fingerprint: 8FA7B29C...
Dialogue tokens: stay, daughter, promise
```

It does not have to store or stream the original film.

The challenge is obtaining and processing reference content legally. You cannot simply download thousands of copyrighted movies from streaming platforms and index them for a commercial application without considering content rights.

Option 3: Hybrid architecture — recommended

The proper full Reelix architecture is hybrid:



The system first searches Reelix's own index. If it cannot find a confident match, it uses external AI.

This produces:

- Fast results for popular indexed titles
- Broader identification through AI
- Lower AI cost
- Better exact-scene matching
- Less dependency on one provider
- Better confidence verification

How similar full apps are built

Their exact backend implementations are private, but commercial movie-recognition systems generally rely on:

1. Video or audio fingerprinting
2. Indexed reference content
3. Computer-vision models

4. Speech-to-text and subtitle search
5. Movie metadata providers
6. Cloud servers and databases

A previously documented movie-identification product combined audio recognition, computer vision and an indexed movie-scene database. It launched with a manageable catalogue and expanded gradually instead of supporting every movie immediately.

 [www.wired.c...](http://www.wired.com)

So similar apps are not performing recognition magically without data. They are normally doing one of the following:

- Maintaining a private recognition index
- Paying for a third-party recognition/AI service
- Combining both approaches
- Supporting only a limited catalogue
- Using user feedback to improve future matches

Will the full app be completely free to operate?

No. A full app with real users cannot realistically operate forever at zero cost.

You can reduce costs heavily using:

- Free database tiers at the beginning
- On-device OCR
- On-device frame extraction
- Small compressed requests
- A fingerprint index instead of full videos
- AI only when local/index matching fails
- Automatic deletion of captured evidence
- Advertisements or subscriptions to cover operating costs

But as usage grows, costs will appear in:

- Server hosting
- Vector/fingerprint search
- AI API usage
- Database storage
- Network bandwidth
- Monitoring and security
- Movie metadata commercial licensing
- Google Play registration and service fees

That is normal. “Nearly zero budget” can get Reelix launched, but revenue must eventually cover each scan.

Screen recordings in the full app

Your storage requirement is completely achievable:

Reelix will not save screen recordings to the Gallery or permanently store them on the phone.

The full application should use this pipeline:

1. User taps the bubble.
2. Screen frames enter a temporary memory buffer.
3. Reelix samples 4–8 useful frames.
4. OCR extracts visible subtitles.
5. A temporary audio buffer is captured where Android allows it.
6. Fingerprints are calculated.
7. Only compressed evidence or fingerprints are sent.
8. The memory buffer is cleared.
9. The result appears.
10. No recording remains.

```
graph TD
    A[Social-media Reel] --> B[5-second RAM buffer]
    B --> C[Frames + text + audio fingerprint]
    C --> D[Recognition server]
    D --> E[RAM cleared]
    E --> F[Result displayed]
```

Android's screen-capture API gives the application access to a media stream; it does not require creating a permanent video in the user's Gallery.  developer.android...

If a device requires temporary encoding, Reelix can use a private cache file and immediately delete it. It will not be visible in the Gallery. Android also recommends that apps actively delete temporary cache files.  developer.android...

Full Reelix backend structure

A commercially viable architecture would look like this:

Component	Responsibility
Flutter app	Interface, history, settings and result screens

Component	Responsibility
Native Kotlin service	Bubble, screen capture and foreground notification
Memory processor	Select frames without permanently saving video
On-device OCR	Extract subtitles and captions
Audio extractor	Capture permitted playback audio
Recognition API	Receive fingerprints and coordinate matching
Vector database	Search visually similar scenes
Dialogue index	Search quotes and subtitle fragments
Metadata database/API	Posters, titles, cast and descriptions
AI fallback	Handle titles not available in the index
Authenticity engine	AI-video likelihood and C2PA checking
Application database	Users, history, feedback and subscriptions

Practical catalogue strategy

The full app should not promise every movie immediately. It should continuously grow its supported catalogue.

A realistic launch catalogue could prioritize:

- Most frequently used Netflix-series clips
- Popular Hollywood films
- Popular Asian dramas
- Anime commonly used in Reels
- Trending films and series
- Titles requested by Reelix users

When identification fails, users can press:

“Request this title”

This gives you demand data showing which content should be indexed next.

Final answer

For the full Reelix app:

- **Yes**, an application database is required.
- **Yes**, reliable exact-scene recognition requires either Reelix's own searchable index or a paid external service.
- **No**, you do not need to store full movies in the database.
- **No**, screen recordings do not need to be saved on users' phones.
- **No**, a production app with growing usage cannot operate permanently for zero cost.
- **Yes**, your exact bubble → temporary capture → analyse → delete → overlay-result experience can be implemented.

The most sustainable full-app design is a **hybrid recognition system**: Reelix's fingerprint index for frequently encountered content, AI fallback for unknown content, and memory-only screen capture on the phone.

ok first what are we gonna do? designing?

Worked for 55s

Not full visual design yet.

First we will create the **Reelix product blueprint**: exactly what the full app does, which screens/features it needs, and the complete user flow. Then we validate whether identification actually works before spending time designing a large app.

Order:

1. Define the full Reelix flow

- Open app
- Enable scanner permission
- Floating bubble appears
- User taps bubble on a Reel
- 5–12-second temporary capture
- Analyse
- Show movie/series result
- Save result / view history
- AI-video analysis result

2. List every screen and feature

- Splash/onboarding
- Home scanner screen
- Permission setup
- Bubble settings

- Scanning state
- Match result
- No-result screen
- History and saved watchlist
- AI-check result
- Profile/settings/privacy

3. Make a simple user-flow diagram and low-fidelity wireframes

- Just layout and navigation first
- Not colours, logos or polished Figma design yet

4. Test the important technical part

- Use around 100 varied, lawful movie/TV Reel clips
- Check whether our identification method gives useful top-1/top-3 results
- Check response time and honest “no match” behaviour

5. Then design the attractive final UI in Figma

- Brand name/logo
- Colours
- Bubble animation
- Result cards
- Full polished screens

So the immediate next task is: **I will give you the complete Reelix feature list and screen-by-screen user flow.** That becomes the foundation for design and coding.

Ok good go for it. also i have antigravity. So if u give exact instuctions. I can easily do this in antigravity

Yes. **We'll use Antigravity to build Reelix step by step, with a clear specification and acceptance checks for each stage.** First, we'll give it the full-app blueprint—not ask it to generate the entire application in one attempt.

One correction to my earlier answers: **a full app does not automatically require your own movie database.** It can depend on external recognition services. We still need to evaluate their accuracy, commercial terms and running costs before choosing the final architecture.

1. Reelix's agreed product scope

Area	Requirement
Platform	Android first
Main interaction	Draggable floating bubble over other apps
Capture	User-triggered, short screen sample
Storage	No permanent recordings or Gallery files
Identification	Movie/TV candidates using visual, dialogue and subtitle evidence
Results	Compact overlay, with an option to open full details
Uncertainty	Possible matches or “Unable to identify”—no invented certainty
AI-content check	Separate experimental assessment, not a guaranteed real/fake verdict
History	Store title/result information, not captured footage
Budget	Minimise costs; no paid services activated without your approval

The floating bubble remains the central feature of the full app. Any technical experiments are development checks, not a replacement for that experience.

2. Screen and interaction blueprint

We should separate full screens from overlays so we don’t create unnecessary pages.

Screen or component	What it does
Welcome/onboarding	Explains scanning and privacy
Home	Start/stop scanner, recent results and scanner status
Permission guidance	Explains permissions before opening Android’s system prompts
Floating bubble	Start scan; expand menu for identification, AI check and stop
Capture overlay	Countdown and Cancel

Screen or component	What it does
Analysis overlay	Progress and Cancel
Result overlay	Likely title, match strength, Details and Dismiss
Uncertain/error overlay	Possible matches, retry, connection or capture problem
Title details	Poster, year, synopsis, evidence and Save
History	Previous identification results; delete controls
Watchlist	Saved titles
AI assessment	Evidence, limitations and inconclusive results
Settings/privacy	Bubble options, history controls, data handling and help

Accounts, subscriptions and cloud synchronisation remain optional decisions. We shouldn't add mandatory login just because this is a full application.

3. What to do in Antigravity now

Open the folder you want to use for Reelix. Paste the following as the first instruction. This task is **planning only**—it should produce documents you can review before code is generated.

You are helping me build Reelix, a complete Android application intended for eventual Google Play publication.

TASK 1: Create the product blueprint and implementation plan.
Do not implement application code yet.

First inspect the workspace. Preserve all existing files and changes.
If a project already exists, describe its structure before proposing changes. Do not overwrite existing planning documents without reviewing them.

PRODUCT

Reelix helps users identify movies and TV series shown in social-media Reels through a floating, draggable Android bubble.

Required user experience:

1. Open Reelix.
2. Enable the scanner after clear privacy disclosure.
3. Grant the necessary Android permissions through real system dialogs.
4. See a floating bubble while using another app.
5. Tap the bubble to collect a short screen sample.

6. See a capture countdown with Cancel.
7. Analyse the sample.
8. See a compact result overlay.
9. Open details, save the title, dismiss, or try again.

The bubble is the primary interaction, not an optional replacement for manual uploads.

CAPTURE AND PRIVACY

- Never save captured recordings to Gallery, Downloads or shared storage.
- Prefer bounded in-memory processing.
- If temporary private cache is technically necessary, explicitly document it, enforce size/time limits, and clean up after success, failure, cancellation and application restart.
- Never retain raw captures as scan history.
- No continuous retained recording while waiting for a tap.
- Respect protected content and unavailable playback audio.
- Do not use Accessibility Service to bypass platform restrictions.
- Explain uploads and third-party processing honestly.
- Do not promise zero provider retention without verifying provider terms.

CRITICAL PLATFORM INVESTIGATION

Verify current official Android documentation before specifying capture.

Distinguish:

- A. Stopping the collection of a short sample.
- B. Ending the MediaProjection session.

Explain how repeated bubble scans work, when renewed consent is required, what stays active between scans, and battery/privacy consequences.

Do not assume one permission grant lasts forever.

Cover screen lock, user revocation, process death and service shutdown.

IDENTIFICATION

Evaluate two full-production architecture options:

1. External recognition/multimodal service without our own scene index.
2. A licensed reference index, potentially with external AI fallback.

Do not assume either option is free, accurate or mandatory.

Movie metadata APIs provide title information; they do not independently prove that a captured scene matches that title.

Use visual frames, subtitle OCR and permitted audio where useful.

Return possible candidates or no result when evidence is insufficient.

Do not invent confidence percentages or exact episode timestamps.

Any numerical confidence requires evaluation and calibration.

AI-CONTENT ASSESSMENT

Treat this as a separate experimental capability.

Include an inconclusive outcome.

Do not label a clip genuine merely because no AI evidence was detected.

Do not assume a screen recording preserves the original video's

Content Credentials or metadata.

BUDGET AND SECURITY

- Near-zero initial budget.
- Do not activate billing, subscriptions, deployments or paid APIs.
- Never put provider secrets in the mobile application or repository.
- Propose a protected backend boundary for secret-bearing API requests.
- Identify commercial licensing, quotas, data-use terms and future costs.
- Do not scrape or download copyrighted movies to build an index.
- Do not add mandatory accounts or payments without a documented need.

DELIVERABLES

Create:

1. docs/product-requirements.md
2. docs/user-flows.md
3. docs/screens-and-states.md
4. docs/architecture-options.md
5. docs/validation-plan.md
6. docs/implementation-roadmap.md

Include:

- Full-app requirements and optional features.
- Screen versus overlay classification.
- Success, error, cancelled, offline and permission-denied states.
- Proposed architecture with unresolved decisions clearly labelled.
- Official source links for platform claims.
- Recognition evaluation using varied, lawfully obtained test clips.
- Separate technical checks for capture, cleanup and recognition.
- Ordered milestones with testable acceptance criteria.

Consider Flutter UI with native Kotlin integration as a candidate, not an already verified final architecture.

Finish by reporting:

- Files created.
- Key unresolved decisions.
- Risks that could change the design.
- The next single implementation task.

Do not generate production code, deploy anything, or claim untested capabilities work.

4. What happens next

After Antigravity finishes, **send me its summary and unresolved decisions**. We'll review those, then move to the screen layouts and technical checks. Once those constraints are

clear, I'll give you the next exact instruction for implementing Reelix.



architecture-options.md

File



implementation-roadm...

File



product-requirements....

File



screens-and-states.md

File



user-flows.md

File



validation-plan.md

File

Worked for 23s

You've hit your usage limit. Upgrade your plan or add credits to continue, or try again at 1:11 PM.

ChatGPT can make mistakes. Check important info.